

C Programming Primer: Basics to Intermediate for Phd students

A compact, practice-oriented guide covering variables, control flow, loops, pointers, and structures—with examples and exercises from beginner to intermediate level.

0) How to compile & run

```
# Using gcc
gcc -Wall -Wextra -O2 hello.c -o hello
./hello
```

1) Hello, World

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0; // success status
}
```

Key ideas: #include pulls in headers, main returns an int, \n ends a line.

Exercise 1.1: Print your name and the current year on separate lines.

2) Basic Types, Variables, and Constants

Primitive integer types: char, short, int, long, long long (plus signed/unsigned).

Floating point: float, double, long double.

Others: _Bool (or bool via <stdbool.h>), size_t, ptrdiff_t.

Declaring variables

```
int age = 30;           // initialization
float price = 9.99f;    // note the 'f' suffix
char initial = 'A';     // single quotes for chars
```

Constants

```
const double PI = 3.141592653589793;
#define MAX_USERS 1000
```

Reading/Writing

```
#include <stdio.h>
```

```
int main(void) {  
    int x; double y; char c;  
    printf("Enter int double char: ");  
    scanf("%d %lf %c", &x, &y, &c); // note: addresses  
    printf("x=%d y=%g c=%c\n", x, y, c);  
    return 0;  
}
```

Exercise 2.1: Compute the area and circumference of a circle from a user-given radius (double). Print with 2 decimals.

3) Operators & Expressions (quick tour)

- Arithmetic: + - * / %
- Increment/decrement: ++ --
- Assignment: = += -= *= /= %=
- Comparison: == != < <= > >=
- Logical: && || !
- Bitwise: & | ^ ~ << >>
- Precedence matters; use parentheses to be safe.

Exercise 3.1: Given integer n, print whether it is even or odd using % and a conditional.

4) Control Statements

if / else

```
if (score >= 90) {  
    puts("A");  
} else if (score >= 80) {  
    puts("B");  
} else {  
    puts("C or below");  
}
```

switch

```
switch (day) {  
    case 1: puts("Mon"); break;  
    case 2: puts("Tue"); break;  
}
```

```
        default: puts("Other"); break;
    }
}
```

Loops: for, while, do-while

```
for (int i = 0; i < 5; ++i) printf("%d ", i);
```

```
int j = 5; while (j > 0) { printf("%d ", j--); }
```

```
int k = 0; do { printf("%d ", k++); } while (k < 3);
```

Exercise 4.1: Print the first n Fibonacci numbers using a for loop.

Exercise 4.2: Make a simple number-guessing game with while and break.

5) Arrays & Strings

Arrays

```
int a[5] = {1,2,3,4,5};
size_t len = sizeof a / sizeof a[0];
```

Strings (null-terminated char arrays)

```
#include <stdio.h>
#include <string.h>

int main(void) {
    char name[32];
    printf("Enter name: ");
    fgets(name, sizeof name, stdin);
    name[strcspn(name, "\n")] = '\0'; // strip newline
    printf("Hello, %s!\n", name);
    return 0;
}
```

Exercise 5.1: Read a line and count vowels (a,e,i,o,u), case-insensitive.

Exercise 5.2: Implement reverse_string(char *s) in place.

6) Functions

```
#include <stdio.h>
```

```
int add(int a, int b) { return a + b; }
```

```
void greet(const char *who) { printf("Hi, %s!\n", who); }
```

```
int main(void) {
    printf("%d\n", add(2,3));
    greet("world");
}
```

Prototypes go before main or in headers.

Exercise 6.1: Write `int is_prime(int n)` and test it.

7) Pointers (core concept)

A pointer stores an address. Use `&` to take the address, `*` to dereference.

```
int x = 42;
int *px = &x;    // px points to x
*px = 100;       // changes x
printf("x=%d\n", x);
```

Pointer arithmetic moves in units of the pointed type.

```
int arr[3] = {10,20,30};
int *p = arr;    // arr decays to &arr[0]
*(p+1) = 99;     // arr[1] = 99
```

Pass-by-pointer to modify callers' data

```
void swap(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
```

Dynamic memory (`malloc`, `calloc`, `realloc`, `free`)

```
#include <stdlib.h>
```

```
int *make_array(size_t n) {
    int *p = malloc(n * sizeof *p);
    if (!p) return NULL; // always check
    for (size_t i=0; i<n; ++i) p[i] = (int)i;
    return p;
}
```

Exercise 7.1: Allocate an array of `n` doubles, fill with random numbers in `[0,1)`, compute mean and variance, then free.

Exercise 7.2: Implement a dynamic stack of `int` using an array that resizes with `realloc`.

8) Structures (struct) and typedef

Structures group related fields.

```
typedef struct {
    int id;
    char name[32];
    double gpa;
} Student;

void print_student(const Student *s) {
    printf("%d %s %.2f\n", s->id, s->name, s->gpa);
}

int main(void) {
    Student a = { .id=1, .name="Ana", .gpa=6.5 };
    print_student(&a);
}
```

Arrays of structs

```
Student cls[2] = { {1, "Ana", 6.5}, {2, "Ben", 5.9} };
```

Pointers to structs: use -> to access members via pointer.

Exercise 8.1: Read *m* students from stdin, then print the top 3 by GPA.

Exercise 8.2: Model a 2D point and a distance(Point a, Point b) function using sqrt.

9) File I/O (basics)

```
#include <stdio.h>
```

```
int main(void) {
    FILE *fp = fopen("data.txt", "w");
    if (!fp) return 1;
    fprintf(fp, "%d %s\n", 42, "answer");
    fclose(fp);

    fp = fopen("data.txt", "r");
    int n; char buf[16];
    fscanf(fp, "%d %15s", &n, buf);
    printf("Read: %d %s\n", n, buf);
    fclose(fp);
}
```

Exercise 10.1: Copy a text file to another, counting lines and bytes.

10) Headers & Modularization

mathx.h

```
#ifndef MATHX_H
#define MATHX_H

double mean(const double *a, int n);

#endif
```

mathx.c

```
#include "mathx.h"

double mean(const double *a, int n) {
    double s = 0.0;
    for (int i=0; i<n; ++i) s += a[i];
    return n ? s/n : 0.0;
}
```

main.c

```
#include <stdio.h>
#include "mathx.h"

int main(void){
    double v[3] = {1,2,3};
    printf("mean=%.2f\n", mean(v,3));
}
```

Build

```
gcc -c mathx.c      # -> mathx.o
gcc -c main.c       # -> main.o
gcc main.o mathx.o -o app
```

11) Common Pitfalls & Best Practices

- Always initialize variables; uninitialized reads are UB (undefined behavior).
- Bounds check: never write past array length; prefer sizeof tricks.
- Check return values (I/O, malloc, scanf).
- Prefer fgets over gets (unsafe). Limit %s widths in scanf/printf.
- Use -Wall -Wextra -Wpedantic and consider sanitizers during development:

```
gcc -fsanitize=address,undefined -fno-omit-frame-pointer -g app.c -o app
```

- Keep headers minimal; avoid global variables; document invariants.

12) Worked Examples

13.3 Tiny CSV Reader into Structs

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int id; char name[32]; double value; } Row;

int main(void){
    FILE *fp = fopen("data.csv", "r");
    if(!fp){ perror("data.csv"); return 1; }
    Row *rows = NULL; size_t n=0, cap=0;
    char line[128];
    while (fgets(line, sizeof line, fp)){
        if (n==cap){ cap = cap? cap*2:8; rows = realloc(rows, cap*sizeof
*rows); }
        sscanf(line, "%d,%31[^\n],%lf", &rows[n].id, rows[n].name,
&rows[n].value);
        ++n;
    }
    fclose(fp);
    for(size_t i=0; i<n; ++i)
        printf("%d %s %.2f\n", rows[i].id, rows[i].name, rows[i].value);
    free(rows);
}
```

14) Progressive Exercise Set (with hints)

Beginner 1. Read an int `n` and print the sum from 1..`n`. (*Loop, int arithmetic*) 2. Read three integers and output the maximum. (*if/else*) 3. Read a line, print it in uppercase. (*fgets, toupper*)

Intermediate A 4. Implement `binary_search(int *a, int n, int key)`. (*arrays, indices*) 5. Implement `string_to_int(const char *s)` handling optional leading +/- and digits. (*pointers*) 6. Build a Complex struct with add, mul, and abs (magnitude). (*structs, sqrt*)

Intermediate B 7. Implement a dynamic vector of double with operations: push, pop, get, set, resize, free. (*malloc/realloc*) 8. Implement a simple linked list of integers with

push_front, insert_after, remove, find, and destroy. (*struct pointers*) 9. Parse a text file of “name score” pairs into an array of structs and print the top-k. (*file I/O, qsort*)

Intermediate C 10. Implement a Matrix type supporting create(r,c), free, get, set, and mul. **(double**, row-major flat arrays)* 11. Write a tiny CLI: *calc op a b* where *op* in {*add, sub, mul, div*}. (*argv/argc parsing, function pointers*) 12. Implement a mini JSON-like parser for {*key:value*} pairs with no nesting. (*strings, state machine*)*

15) Reading List & Next Steps

- *The C Programming Language* (Kernighan & Ritchie, 2nd ed.)
 - *C Programming: A Modern Approach* (K. N. King, 2nd ed.)
-